사람의, 사람에 의한, 사람을 위한

이연준, KAIST 전산학부

사람의, 사람에 의한, 사람을 위한. 이는 오늘날 우리가 어떠한 자세로 개발에 임해야 하는지 보여준다. 오늘날, 대형 언어 모델(LLM, Large Language Model)의 발전과 함께 많은 개발자가 코드에 대한 이해 없이 지시(Prompting)만으로 올바른 소프트웨어를 만들고자 한다. 하지만 이러한 시도는 오히려 올바르지 않은 소프트웨어를 양산하는 결과를 가지고 왔다. 본 에세이에서는 올바른 소프트웨어를 개발하기 위해, 대형 언어 모델을 지혜롭게 활용하는 방안에 대하여 논의한다.

모든 소프트웨어를 혼자만의 힘으로 작성할 수 있는가? 아마 현대의 많은 개발자들은 대형 언어 모델의 도움 없이는 모든 것을 구현하기 힘들 것이다. 대형 언어 모델이 발전함에 따라 개발자가 직접 모든 기능을 구현할 필요가 사라졌다. 나 역시도 소프트웨어를 개발할 때 주석으로 무엇을 구현해야 할지 상세히 설명하고, 코덱스(Codex)가 코드를 생성하길 기다리곤 한다. 조금만 기다리면 코덱스는 수정 기록을 보여주며 내가 구현하고자 하는 코드를 생성해 준다. 개발 과정에서 대형 언어 모델의 사용이 증가하는 가장 근본적인 원인은 오늘날, 소프트웨어가 방대해지며, 개발자가 모든 코드를 분석하는 것이 어려워졌기 때문이다.

소프트웨어가 방대해지며 소프트웨어의 올바름이란 개념이 화두로 떠올랐다. 올바른 소프트웨어는 사전에 정의된 명세를 만족하며, 어떠한 오류도 발생하지 않고, 원래의 의도대로 정확히 동작하는 소프트웨어를 의미한다. 하지만 매년 새롭게 추가되는 보안 취약점 식별자(CVE, Common Vulnerabilities and Exposures) [1]를 보면, 올바르지 않은 소프트웨어는 계속 증가하고 있다. 이는 소프트웨어의 올바름을 검증하는 것이 매우 어려움을 나타낸다. 아무리 간단한 소프트웨어일지라도, 소프트웨어의 동작은 내부 실행 환경에 큰 영향을 받고, 가능한 입력이 거의 무한하기 때문이다. 이러한 이유로 개발자들은 테스트와 정적 분석으로 올바른 소프트웨어를 만들고자 노력한다.

하지만 이러한 노력에 무색하게 대형 언어 모델의 발전은 소프트웨어의 올바름 검증을 더욱 어렵게 만들었다. 앞서 언급했듯, 대형 언어 모델의 발전은 개발을 더욱 편리하게 만들어주며 개발 방식을 변화시켰다. 하지만, 이와 동시에 개발자가 대형 언어 모델에 크게 의존하는 문제점도 발생하였다. 이는 개발자가 대형 언어 모델의 제안을 곧이곧대로 받아들이면서 심화되었다. 일례로, 오류 수정까지 대형 언어 모델에 맡기며, 대형 언어 모델이 개발과 코드 수정을 모두 수행하는 현상까지 발생하였다. 대형 언어 모델의 발전은 편리한 개발을 가져오는 것뿐만 아니라, 개발자(Programmer)를 더 이상 코드를 이해하지 않고 단순히 지시문(Prompt)만 작성하는, "지시자(Prompter)"로 퇴화시키는 악영향도 가져왔다.

혹자는 오히려 대형 언어 모델이 개발과 코드 수정을 모두 수행하면 최고의 결과를 도출할 것이라고 주장한다. 하지만, 이는 최고의 결과처럼 보이는 것일 뿐임을 알아야 한다. 대형 언어 모델은 이전까지의 자료에 기반해 결과를 생성한다. 즉, 현재 개발자의 의도에 맞는 코드보다는 기존에 자주 사용하는 방식으로 코드를 생성한다. 실제로, 내가 각종 대형 언어 모델을 중계하는 서버(Server)를 구축할 때, 대형 언어 모델은 잘못된 코드를 당당히 제안하였다. 회사마다 다른 나들목(API, Application Programming Interface)을 사용해야 했으나, 대형 언어 모델은 각 회사의 나들목 정보를 사용하지 않고, 그저 많은 사람이 사용한 나들목을 사용하였다. 이처럼 대형 언어 모델은 알맞은 것처럼 보이는 코드를 생성할 뿐, 의도에 완전히 부합하는 코드를 생성하지는 않는다.

이러한 이유로 구글(Google) [3]과 페이스북(Facebook) [2]은 소프트웨어를 개발할 때 항상 개발자의 최종 검토(code review) 과정을 거친다. 이는 대형 언어 모델을 활용한 개발 방식의 장점인 편리한 개발을 침해하지 않으면서도, 개발 의도에 맞게 코드를 작성하며 올바른 소프트웨어를 만드는 데 일조한다. 대형 언어 모델이 주는 개발의 편리함은 무시할 수 없는 장점이기에, 우리는 대형 언어 모델을 사용하면서도 올바른 소프트웨어를 만들고자 노력해야 한다.

대형 언어 모델이 소프트웨어의 올바름에 악영향을 끼치지 않도록 하려면 개발자는 사람의, 사람에 의한, 사람을 위한 개발을 수행해야 한다. 가장 먼저 사람의 개발은 대형 언어 모델이 아닌 사람이 주체가 되어 개발해야 한다는 것으로, 지시자가 아니라 개발자가 되어야 한다는 것이다.

이를 위해 개발자는 대형 언어 모델을 정답지가 아닌 도구에 불과하다는 생각을 가지고, 자신이 추가한 코드에 책임을 져야 한다. 코드를 작성하기만 하는 코더에서 넘어가서, 코드가 어떤 역할을 수행하고, 왜 이런 방식으로 구현되어야 하는지 항상 이해하고 있어야 한다. 작성한 코드에 대해 이해하고 있을 때, 비로소 올바른 소프트웨어를 개발할 준비가 되었다고 할 수 있다.

사람에 의한 개발은 모든 개발 과정의 최종 확인 및 검토는 사람이 수행해야 한다는 것으로, 이는 항상 의도에 맞는 코드를 작성하도록 도와준다. 이를 위해 개발자는 대형 언어 모델의 결과를 그대로 받아들이는 것이 아니라, 대형 언어 모델이 왜 해당 코드를 제안했는지, 정말로 해당 코드가 최적의 코드인지 항상 고민하고 비판적으로 생각하는 자세를 가져야 한다. 비판적인 생각이 기본이 될 때, 개발자는 더욱 올바른 소프트웨어를 개발해 나갈 수 있다.

사람을 위한 개발은 정적 분석기나 대형 언어 모델과 같이 개발 및 오류 수정의 편리함을 주는 도구를 도구로서 지혜롭게 사용해야 함을 의미한다. 올바른 소프트웨어를 위해서는 오류가 발생하지 않도록 개발하는 것이 중요한데, 이는 개발자 혼자서 수행하기에는 매우 어려운 작업이다. 이때, 대형 언어 모델에 오류에 대한 설명과 수정 방안을 요청하여 오류 수정에 도움받을 수 있다. 하지만 대형 언어 모델의 결과를 그저 도구의 추천이라 생각하고, 틀릴 수 있음을 인지해야 한다. 정적 분석기로 오류를 검출하고, 대형 언어 모델로 수정 방안을 제안받으며 우리는 더 편리하게 개발을 지속해 나갈 수 있다.

완벽하게 올바른 소프트웨어를 작성하는 것은 사실상 불가능한 일이다. 하지만, 그렇다고 해서 소프트웨어를 올바르게 만드는 노력이 헛된 것은 아니다. 이 과정에서 치명적인 오류를 발견할 수도 있고, 코드의 가독성을 향상시킬 수도 있다. 이를 위해 우리는 계속 소프트웨어를 올바르게 만들고자 노력해야 한다. 당연히 이는 지시자가 아닌 사람의, 사람에 의한, 사람을 위한 개발을 할 때 달성할 수 있는 목표이다.

References

[1] Alan. 2026. *Hundreds of new vulnerabilities a day: Web security is changing.* SiteHost. https://sitehost.nz/blog/hundreds-new-vulnerabilities Accessed: 2026-03-04.

[2] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O'Hearn. 2019. Scaling static analyses at Facebook. *Commun. ACM* 62, 8 (July 2019), 62–70. doi:10.1145/3338112

[3] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan. 2018. Lessons from building static analysis tools at Google. *Commun. ACM* 61, 4 (March 2018), 58–66. doi:10.1145/3188720